

SHOPLOC — GUIDE D'INSTALLATION DE L'OUTILLAGE LOGICIEL & QUESTIONNAIRE D'ARBITRAGE MOA

Master 2 MIAGE — UE Génie Logiciel par la Pratique 2026-2027 — Jalon R2

Intitule du Projet	Plateforme ShopLoc — Marketplace & Fidelisation Territoriale
Identifiant Officiel	MiageShopLoc
Titre du Document	ShopLoc — Guide d'Installation de l'Outillage Logiciel & Questionnaire d'Arbitrage MOA
Reference Documentaire	GLOP-2026-DEV-GUIDE-OUTILS-R2-v1.0
Contexte Academique	Master 2 MIAGE — UE Genie Logiciel par la Pratique (2026-2027)
Maitrise d Ouvrage (MOA)	Laurence Duchien, Anne Etien, Francois Secchi, Jeremy Woirhaye
Entreprise Soumissionnaire	Garik (Equipe de 5 etudiants Master 2 MIAGE)
Date de remise	18 Septembre 2026 (18h00)
Statut du document	VERSION 1.0 — FORMELLE
Assistance IA & Transparence	Document élaboré avec l'assistance d'une intelligence artificielle (Antigravity), sous la direction, la relecture critique et la validation intégrale de l'équipe d'ingénierie Garik.

Table des Matières

GUIDE D'INSTALLATION, JUSTIFICATIONS TECHNIQUES & QUESTIONNAIRE D'ARBITRAGE MOA (JALON R2)

Cartouche d'Identification Documentaire

1. Contexte & Objectif Opérationnel de la Séance du 28 Septembre 2026

- 1.1. Transition du Jalon R1 vers le Jalon R2
- 1.2. Objectif de la Séance Présentielle du 28 Septembre 2026

2. Cartographie Technique & Justification Approfondie des Outils Retenus

- 2.1. Gestionnaire de Code & Forge Logicielle : GitLab (Université de Lille)
- 2.2. Langage & Framework Backend : Java 21 LTS + Spring Boot 3.3
- 2.3. Gestionnaire de Build & Dépendances : Apache Maven 3.9+
- 2.4. Framework Frontend & Ergonomie : React 18 + TypeScript 5 + Tailwind CSS 3
- 2.5. Base de Données Relationnelle : PostgreSQL 16
- 2.6. Conteneurisation & Orchestration Locale : Docker & Docker Compose v2
- 2.7. Simulateurs de Services Tiers : Mocks REST OpenAPI 3.1 Conteneurisés
- 2.8. Qualimétrie, Tests & Barrière de Qualité : JUnit 5, JaCoCo & SonarQube

3. Guide d'Installation Pas-à-Pas Multi-OS (Windows, macOS, Linux)

- 3.1. Gestionnaire de Versions Git & Clé SSH GitLab Univ-Lille
- 3.2. Kit de Développement Java 21 LTS (OpenJDK Temurin)
- 3.3. Gestionnaire de Build Apache Maven 3.9+
- 3.4. Écosystème Frontend : Node.js 20+ LTS & npm
- 3.5. Conteneurisation : Docker Desktop / Docker Engine
- 3.6. Environnements de Développement Intégrés (IDE) & Extensions

4. Protocole de Validation Globale du Poste de Développement

- 4.1. Tableau de Contrôle Diagnostic en 5 Commandes
- 4.2. Matrice des Résultats Attendus

5. Questionnaire d'Arbitrage MOA : Les Questions Stratégiques à Poser le 28/09

- 5.1. Volet A : Cadrage du Mandat J2E & Validation Formelle de Spring Boot 3.3
- 5.2. Volet B : Infrastructure CI/CD sur la Forge GitLab de l'Université de Lille
- 5.3. Volet C : Qualimétrie, SonarQube & Couverture de Tests
- 5.4. Volet D : Conception et Hébergement des Simulateurs Partenaires (Mocks REST)
- 5.5. Volet E : Modalités d'Évaluation et Livrable Exact du Jalon R2 (12 Octobre 2026)
- 5.6. Volet F : Site de Suivi de Projet & Reporting des Temps d'Activité

6. Organisation et Déroulement de l'Équipe en Séance

- 6.1. Répartition des Prises de Parole en Séance
- 6.2. Directives de Posture Professionnelle
- 6.3. Actions Immédiates Post-Séance

GUIDE D'INSTALLATION, JUSTIFICATIONS TECHNIQUES & QUESTIONNAIRE D'ARBITRAGE MOA (JALON R2)

Cartouche d'Identification Documentaire

Propriété	Définition & Valeur Officielle
Intitulé du Projet	Plateforme Territoriale ShopLoc — Marketplace & Fidélisation
Identifiant Officiel	MiageShopLoc
Titre du Document	Guide d'Installation de l'Outillage Logiciel, Justifications Techniques & Questionnaire d'Arbitrage MOA
Référence Documentaire	GLOP-2026-DEV-GUIDE-OUTILS-R2-v1.0
Date d'Émission	27 Septembre 2026
Statut du Document	Validé — Référentiel interne d'ingénierie et de préparation de séance
Équipe Prestataire	Entreprise Garik (5 élèves-ingénieurs M2 MIAGE, Université de Lille)
Rédacteur Principal	Rayane Alli (Spécialiste Outils & Ingénieur Logiciel / Scrum Master Sprint 1)
Relecteurs & Validateurs	Abdelkader Heddi (Communication), Khalil Bouchama (QA), Gautam Demeulemeester (Back), Ilyas Ait Ali (Front)
Maîtrise d'Ouvrage (MOA)	Laurence Duchien, Anne Etien, François Secchi, Jérémy Woirhayé
Dépôt GitLab Évalué	git@gitlab-ssh.univ-lille.fr:khalil.bouchama.etu/projet-glop-app.git

1. CONTEXTE & OBJECTIF OPÉRATIONNEL DE LA SÉANCE DU 28 SEPTEMBRE 2026

1.1. Transition du Jalon R1 vers le Jalon R2

Le premier jalon contractuel R1 (Cahier des charges et Cadrage) a été remis le 18 septembre 2026 et soutenu le 21 septembre 2026. La maîtrise d'ouvrage (MOA) a confirmé les orientations fonctionnelles et acté le recentrage de la Release 1 de décembre sur l'architecture globale et la réalisation d'un composant vertical complet (Composant de Commande et Panier Multi-Commerces Click & Collect sous protocole Two-Phase Commit, acté dans l'ADR-020).

Le **Jalon R2**, fixé au **lundi 12 octobre 2026**, a pour objectif formel selon le sujet officiel :

» « Choisir, justifier et installer l'ensemble des outils logiciels nécessaires pour réaliser le projet. »

1.2. Objectif de la Séance Présentielle du 28 Septembre 2026

La séance du lundi 28 septembre 2026 est la première des deux séances d'encadrement en présentiel dédiées aux outils logiciels (Séances 4 et 5). L'équipe pédagogique met à disposition un enseignant spécialisé dans le génie logiciel expérimental, l'outillage DevOps et la conteneurisation.

L'objectif impératif de notre équipe pour cette séance est triple :

1. **Zéro temps perdu en installation basique** : Chaque membre doit se présenter en salle avec une machine configurée, testée et prête à compiler.
2. **Arbitrage définitif de l'architecture logicielle** : Lever immédiatement les ambiguïtés contractuelles (notamment la conformité formelle de Spring Boot 3.3 vis-à-vis du mandat J2E énoncé dans le sujet) afin de verrouiller la pile technique sans risque de pivot ultérieur.
3. **Validation de l'infrastructure CI/CD et des simulateurs** : Valider les modalités d'exécution des pipelines sur la forge GitLab de l'Université de Lille (Docker-in-Docker, runners partagés) et le principe de conception des mocks partenaires (banque, transport, voirie).

2. CARTOGRAPHIE TECHNIQUE & JUSTIFICATION APPROFONDIE DES OUTILS RETENUS

Chaque choix technologique a été arrêté sur la base d'une analyse multicritère tenant compte des exigences formelles du sujet, de la robustesse industrielle, de l'éco-conception logicielle et de la maîtrise par les membres de l'équipe.

2.1. Gestionnaire de Code & Forge Logicielle : GitLab (Université de Lille)

- **Technologie retenue** : GitLab Community Edition (Forge institutionnelle de l'Université de Lille).
- **Alternatives analysées et écartées** :
 - *GitHub Enterprise Public* : Écarté pour le code évalué afin de respecter l'obligation de souveraineté et d'accès direct réservé aux enseignants de l'université.
 - *Bitbucket / Gitea autonome* : Écarté en raison de la charge d'administration d'infrastructure superflue.
- **Justification d'ingénierie** :
 - Centralisation académique : Permet aux enseignants d'auditer l'historique des commits, la régularité des contributions individuelles et les revues de code.
 - CI/CD intégrée : Exécution native des pipelines via `.gitlab-ci.yml` sans nécessiter de serveur tiers (Jenkins autonome).
 - Étanchéité de l'architecture bi-dépôt : Le dépôt GitLab `projet-glop-app` reste strictement réservé au code source pur (zéro trace d'outil d'assistance, zéro prompt, zéro fichier interne de gouvernance).
- **Stratégie de branches et convention de commits** :
 - Branches protégées : `main` (production stable), `develop` (intégration continue).
 - Branches de fonctionnalités : `feature/` (fusion via Merge Request avec relecture obligatoire par le Responsable Qualité).
 - Convention Conventional Commits : `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.

2.2. Langage & Framework Backend : Java 21 LTS + Spring Boot 3.3

- **Technologie retenue** : Java 21 LTS avec le framework Spring Boot 3.3 (Spring Data JPA, Spring Security 6, Jakarta Validation, Spring Web).
- **Alternatives analysées et écartées** :
 - *Jakarta EE 10 sous serveur d'application d'entreprise (WildFly 31, Payara 6, TomEE)* : Écarté en raison de la lourdeur d'empreinte mémoire (> 1 Go de RAM au repos), de la lenteur de démarrage et de la complexité de déploiement en conteneur.
 - *Node.js / NestJS (TypeScript)* : Très performant en I/O non bloquant, mais écarté car le sujet stipule formellement : « Le système d'information utilise principalement la technologie J2E pour (i) la définition des composants fonctionnels et (ii) la persistance des données ».

- **Justification d'ingénierie :**

- Conformité au mandat entreprise : Spring Boot s'appuie nativement sur les spécifications Jakarta EE modernes (Jakarta Persistence / Hibernate 6, Jakarta Validation, Jakarta Servlet).
- Intégrité transactionnelle stricte : Indispensable pour l'orchestration du panier multi-commerces (protocole 2PC) et l'incrémentation sûre des points de fidélité via `@Transactional(isolation = Isolation.READ_COMMITTED)`.
- Sécurité et gestion des habilitations : Prise en charge native du contrôle d'accès basé sur les rôles (RBAC) pour les 4 profils actés dans l'ADR-020 (`ROLE_SUPER_ADMIN`, `ROLE_CITY_ADMIN`, `ROLE_ASSOCIATION_ADMIN`, `ROLE_MERCHANT_ADMIN`).
- Frugalité & Green IT : Java 21 avec ramasse-miettes ZGC/G1 et temps de boot optimisé garantit une consommation mémoire maîtrisée en environnement conteneurisé.

2.3. Gestionnaire de Build & Dépendances : Apache Maven 3.9+

- **Technologie retenue :** Apache Maven avec encapsulation par Maven Wrapper (`./mvnw`).
- **Alternatives analysées et écartées :**
 - *Gradle* : Bien que plus rapide grâce à son démon et son cache incrémental, la flexibilité de ses scripts en Groovy/Kotlin introduit des disparités de configuration et complexifie la standardisation des builds sur les postes étudiants.
- **Justification d'ingénierie :**
 - Déterminisme et convention : Configuration déclarative normalisée (`pom.xml`) éliminant les comportements imprévisibles entre environnements.
 - Intégration outillée standard : Plugins officiels de qualité logicielle (JaCoCo pour la couverture de tests, SonarQube Scanner, Maven Surefire pour JUnit 5, Checkstyle) préconfigurés et sans dépendance exotique.
 - Portabilité via Wrapper : Le script `./mvnw` assure que chaque équipier et le runner GitLab CI utilisent strictement la même version de Maven (3.9.x), sans exiger d'installation globale sur l'hôte.

2.4. Framework Frontend & Ergonomie : React 18 + TypeScript 5 + Tailwind CSS 3

- **Technologie retenue :** React 18 en Single Page Application (SPA) configurée en Progressive Web App (PWA), langage TypeScript 5 et stylisation Tailwind CSS 3.
- **Alternatives analysées et écartées :**
 - *Angular 17* : Trop verbeux pour une petite équipe, courbe d'apprentissage abrupte sur le semestre.
 - *Vue.js 3* : Excellent mais typage TypeScript moins strict et écosystème de composants accessibles plus restreint.
 - *Application Mobile Native (Android/iOS)* : Exclue car imposerait le téléchargement sur store, excluant de fait les usagers âgés ou réfractaires (persona Pierre, 74 ans) et multipliant la charge de maintenance.
- **Justification d'ingénierie :**
 - Approche PWA universelle : Accessible depuis tout navigateur mobile ou poste fixe sans installation, installable sur l'écran d'accueil d'un smartphone d'un simple clic.
 - Prise en charge des deux cibles d'ergonomie antinomiques :
 - *Pierre (74 ans, senior)* : Typographie à fort contraste, interface épurée sans navigation imbriquée, affichage direct du pass QR physique/virtuel conforme RGAA niveau AA.
 - *Suzanne (22 ans, commerçante)* : Interface point de vente (POS) ultra-rapide pour validation des retraits Click & Collect et scan caisse en moins de 3 secondes.
 - Typage strict TypeScript : Partage des interfaces de données (DTOs) générées depuis la spécification OpenAPI du backend, interdisant toute divergence d'API.

2.5. Base de Données Relationnelle : PostgreSQL 16

- **Technologie retenue :** PostgreSQL 16 conteneurisé.

- **Alternatives analysées et écartées :**
 - *MySQL 8 / MariaDB* : Moins rigoureux sur la conformité stricte SQL, gestion moins avancée des verrous concurrents fins.
 - *MongoDB / bases NoSQL documentaires* : Totalelement inadaptées aux règles comptables et au clearing financier qui exigent une intégrité référentielle absolue et des garanties ACID sans faille.
- **Justification d'ingénierie :**
 - Propriétés ACID complètes : Isolation stricte lors des réservations concurrentes de stocks sur plusieurs boutiques simultanées.
 - Support hybride relationnel et JSONB : Permet de stocker de manière normalisée les entités pivots (commerces, usagers, transactions) tout en offrant la flexibilité requise pour les attributs variables d'articles (allergènes, calibres, poids).
 - Capacités analytiques pour la collectivité : Moteur de requêtes d'agrégation performant pour calculer les tableaux de bord municipaux de fréquentation k-anonymisés (Marius, DSI Mairie) sans impacter les transactions opérationnelles.

2.6. Conteneurisation & Orchestration Locale : Docker & Docker Compose v2

- **Technologie retenue** : Docker Engine avec Dockerfiles multi-stage et orchestration par Docker Compose v2.
- **Alternatives analysées et écartées :**
 - *Installation native de chaque brique sur l'hôte* : Rejetée catégoriquement en raison des risques de conflits de versions de Java, Node, bases de données entre les différents systèmes d'exploitation de l'équipe (macOS, Windows, Linux).
 - *Kubernetes (k8s / k3s)* : Surdimensionné pour la phase de prototypage, complexité d'exploitation injustifiée.
- **Justification d'ingénierie :**
 - Reproductibilité universelle en commande unique : Tout examinateur ou membre de l'équipe lance l'intégralité du système (backend, frontend, base de données, mocks) via :

```
docker compose up --build -d
```
 - Isolation réseau étanche : Utilisation de réseaux bridge Docker dédiés (shoploc-backend-net, shoploc-db-net) empêchant toute exposition inutile des ports de persistance.
 - Multi-stage build : Génération d'images de production légères (< 200 Mo pour le backend sur image de base Eclipse Temurin JRE Alpine) pour limiter l'empreinte disque et respecter les principes d'éco-conception logicielle.

2.7. Simulateurs de Services Tiers : Mocks REST OpenAPI 3.1 Conteneurisés

- **Technologie retenue** : Simulateurs applicatifs légers (Node.js/Express ou WireMock conteneurisés) exposant des contrats d'interface formalisés OpenAPI 3.1.
- **Alternatives analysées et écartées :**
 - *Appel direct à des APIs externes réelles* : Impossible techniquement (la DSI de Lille, Ilévia et la passerelle bancaire n'ouvrent pas leurs environnements de production aux projets étudiants).
 - *Mocks unitaires Java in-memory (Mockito uniquement)* : Insuffisant car n'éprouve pas la pile réseau, les timeouts et la résilience HTTP réelle du système.
- **Justification d'ingénierie :**
 - Émulation des 3 systèmes externes indispensables :
 1. **Mock Passerelle Bancaire (Izli / Carte Bancaire)** : Simulation de l'autorisation, de la capture globale du panier et des scénarios de rollback 2PC en cas d'échec sur une des boutiques du panier.
 2. **Mock Stationnement Voirie (Horodateurs municipaux)** : Simulation de vérification du droit de stationnement (franchise de 20 minutes accordée aux usagers réguliers VFP) par requête sur plaque d'immatriculation.
 3. **Mock Réseau de Transports Urbains (Ilévia Pass Pass)** : Simulation d'émission et de validation de titres de bus dématérialisés pour les usagers réguliers.
 - Injection de pannes et résilience : Possibilité de simuler des latences réseau, des erreurs HTTP 503 ou des rejets de transaction pour éprouver le comportement dégradé du backend.

2.8. Qualimétrie, Tests & Barrière de Qualité : JUnit 5, JaCoCo & SonarQube

- **Technologie retenue** : JUnit 5, Mockito, AssertJ, plugin JaCoCo et SonarQube Scanner.
- **Justification d'ingénierie** :
 - Test-Driven Development (TDD) : Obligatoire pour les algorithmes sensibles (calcul de la régularité VFP sur fenêtre glissante de 15 jours, transaction 2PC de réservation, clearing des avoirs commerçants).
 - Suivi continu de la dette technique : SonarLint intégré aux IDE pour interception des anomalies à la saisie, puis barrière de qualité (Quality Gate) bloquante sur le pipeline GitLab CI avec un objectif d'au moins 80 % de couverture sur le domaine métier.

3. GUIDE D'INSTALLATION PAS-À-PAS MULTI-OS (WINDOWS, MACOS, LINUX)

Ce guide détaille les procédures exactes d'installation pour chaque membre de l'équipe Garik. Chaque équipier doit exécuter la section correspondant à son système d'exploitation.

3.1. Gestionnaire de Versions Git & Clé SSH GitLab Univ-Lille

Installation du binaire Git

- **macOS** : `xcode-select --install` ou via Homebrew : `brew install git`
- **Windows** : Télécharger l'installateur officiel sur git-scm.com ou via invite de commande administrateur :
`winget install --id Git.Git -e --source winget`
- **Linux (Ubuntu/Debian)** :
`sudo apt update && sudo apt install -y git`

Génération de la clé SSH et association au compte GitLab

Chaque membre doit impérativement associer sa clé SSH à son profil GitLab Université de Lille :

```
# 1. Generation de la cle ED25519 officielle
ssh-keygen -t ed25519 -C "prenom.nom.etu@univ-lille.fr" -f ~/.ssh/id_ed25519_glop

# 2. Ajout de la configuration SSH (~/.ssh/config)
cat << 'EOF' >> ~/.ssh/config
Host gitlab-ssh.univ-lille.fr
  HostName gitlab-ssh.univ-lille.fr
  User git
  IdentityFile ~/.ssh/id_ed25519_glop
  IdentitiesOnly yes
EOF

# 3. Copie de la cle publique pour injection sur GitLab
# macOS : pbcopy < ~/.ssh/id_ed25519_glop.pub
# Windows : clip < ~/.ssh/id_ed25519_glop.pub
# Linux : cat ~/.ssh/id_ed25519_glop.pub
```

Action sur l'interface GitLab : Aller dans *Settings* > *SSH Keys*, coller la clé publique et valider.

Tester immédiatement la connexion :

```
ssh -T git@gitlab-ssh.univ-lille.fr
```

```
# Reponse attendue : Welcome to GitLab, @prenom.nom.etu!
```

3.2. Kit de Développement Java 21 LTS (OpenJDK Temurin)

La version de référence retenue pour notre projet est **Java 21 LTS** (standard de support pour Spring Boot 3.3).

Installation par système d'exploitation

- **macOS :**
`bash
brew install openjdk@21
sudo ln -sf /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk-21.jdk
`
- **Windows :**
`powershell
winget install EclipseAdoptium.Temurin.21.JDK
`
- **Linux (Ubuntu/Debian) :**
`bash
sudo apt update && sudo apt install -y openjdk-21-jdk
`

Vérification impérative

```
java -version  
javac -version  
# Sortie attendue : openjdk version "21.0.x" ou java version "21.0.x"
```

3.3. Gestionnaire de Build Apache Maven 3.9+

- **macOS :** brew install maven
- **Windows :** winget install Apache.Maven
- **Linux (Ubuntu/Debian) :** sudo apt install -y maven

Vérification

```
mvn -version  
# Sortie attendue : Apache Maven 3.9.x
```

3.4. Écosystème Frontend : Node.js 20+ LTS & npm

L'environnement frontend exige Node.js en version Active LTS (v20 ou v22).

- **macOS (via nvm recommandé) :**
`bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
nvm install 20
nvm use 20
`

- **Windows** : Télécharger l'installateur Node.js LTS sur nodejs.org ou exécuter :

```
`powershell
winget install OpenJS.NodeJS.LTS
`
```

- **Linux (Ubuntu/Debian)** :

```
`bash
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
`
```

Vérification

```
node -v
npm -v
# Sortie attendue : v20.x.x ou supérieur / npm 10.x.x ou supérieur
```

3.5. Conteneurisation : Docker Desktop / Docker Engine

- **macOS & Windows** :

- Télécharger et installer **Docker Desktop** depuis docker.com.
- *Sur Windows* : S'assurer que le backend WSL2 (Windows Subsystem for Linux) est activé dans les réglages de Docker Desktop.
- *Sur macOS* : Autoriser l'accès aux sockets d'administration dans les réglages avancés.

- **Linux (Ubuntu/Debian)** :

```
`bash
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER
`
```

Vérification (Docker Desktop doit impérativement être démarré)

```
docker --version
docker compose version
docker run --rm hello-world
# Sortie attendue : Hello from Docker!
```

3.6. Environnements de Développement Intégrés (IDE) & Extensions

Chaque membre équipe son poste de travail des extensions suivantes pour garantir l'homogénéité du code.

Configuration Backend (IntelliJ IDEA ou VS Code)

1. **IntelliJ IDEA Ultimate** (recommandé pour Java/Spring) :

- Téléchargeable gratuitement avec la licence étudiante de l'Université de Lille (compte JetBrains avec email @univ-lille.fr).
 - Extensions à activer : *Spring Boot*, *Database Tools and SQL*, *Docker*, *SonarLint*.

2. Visual Studio Code (si préféré) :

- Pack d'extensions obligatoire : *Extension Pack for Java* (Microsoft), *Spring Boot Extension Pack* (VMware).
 - Extension de qualimétrie : *SonarLint* (SonarSource).

Configuration Frontend (VS Code)

- Extensions obligatoires :
 - *Tailwind CSS IntelliSense* (autocomplétion des classes utilitaires).
 - *ESLint* et *Prettier - Code formatter* (formatage automatique à la sauvegarde).

Outils Clients Complémentaires (Optionnels mais recommandés)

- **Client Base de Données SQL** : **DBeaver Community** (multi-plateforme, gratuit et léger pour inspecter PostgreSQL).
- **Client API REST** : **Postman** ou **Insomnia** (ou extension VS Code *Thunder Client* / *REST Client*) pour exécuter les requêtes sur le backend et les mocks.

4. PROTOCOLE DE VALIDATION GLOBALE DU POSTE DE DÉVELOPPEMENT

Avant la séance de demain matin, chaque membre doit exécuter ce script de contrôle dans son terminal.

4.1. Tableau de Contrôle Diagnostic en 5 Commandes

```
# Etape 1 : Diagnostic des runtimes et compilateurs
echo "=== 1. VERIFICATION OUTILS DE BUILD & RUNTIMES ==="
git --version
java -version
mvn -version
node -v
npm -v

# Etape 2 : Diagnostic de la conteneurisation
echo "=== 2. VERIFICATION DOCKER ==="
docker info | grep "Server Version"
docker compose version

# Etape 3 : Diagnostic de la liaison avec la forge GitLab
echo "=== 3. VERIFICATION ACCES FORGE GITLAB UNIV-LILLE ==="
ssh -T git@gitlab-ssh.univ-lille.fr

# Etape 4 : Test de clonage et de statut du depot applicatif
echo "=== 4. TEST DU DEPOT APPLICATIF ==="
cd /chemin/vers/projet-glop-app
git status
git log -n 2 --oneline
```

4.2. Matrice des Résultats Attendus

Composant	Commande de Test	Résultat Nominal Attendu	Conduite si Erreur
Git	<code>git --version</code>	<code>git version 2.x.x</code>	Installer Git ou vérifier le PATH.
Java	<code>java -version</code>	Version 21.0.x (ou compatible)	Définir la variable JAVA_HOME.
Maven	<code>mvn -version</code>	Apache Maven 3.9.x	Réinstaller via le gestionnaire de paquets.
Node.js	<code>node -v</code>	v20.x.x ou supérieur	Mettre à jour Node.js via nvm ou installateur.
Docker	<code>docker compose version</code>	Docker Compose version v2.x.x	Lancer Docker Desktop et attendre l'icône verte.
SSH Forge	<code>ssh -T git@gitlab-ssh...</code>	Welcome to GitLab, @login!	Vérifier la clé publique sur le profil GitLab.

5. QUESTIONNAIRE D'ARBITRAGE MOA : LES QUESTIONS STRATÉGIQUES À POSER LE 28/09

Ce questionnaire constitue le cœur de notre échange avec l'enseignant expert en outillage lors de la séance du 28 septembre. Les questions sont classées par domaine d'ingénierie et formulées pour obtenir des réponses binaires ou immédiatement opérationnelles.

5.1. Volet A : Cadrage du Mandat J2E & Validation Formelle de Spring Boot 3.3

Le sujet mentionne en page 6 : « Le système d'information utilise principalement la technologie J2E pour (i) la définition des composants fonctionnels et (ii) la persistance des données ».

- **Question A.1 (Fondamentale) :**

« Notre équipe a retenu l'architecture Java 21 avec Spring Boot 3.3, s'appuyant sur Jakarta Persistence (Hibernate 6) et Spring Security 6. Spring Boot est-il formellement validé pour satisfaire l'exigence "J2E" de l'UE, ou attendez-vous impérativement le déploiement d'un serveur d'application Jakarta EE pur (ex. WildFly 31, Payara 6, TomEE) exploitant EJB 3.x et CDI ? »

Objectif : Verrouiller définitivement le choix sans risquer une pénalité d'architecture au jalon R4.

- **Question A.2 (Modularité d'architecture) :**

« Privilégiez-vous pour le backend une structure de projet multi-modules Maven (ex: shoploc-core, shoploc-api, shoploc-fidelity, shoploc-order) au sein d'un monolithe modulaire conteneurisé, ou recommandez-vous un module unique organisé par packages métier (Controller / Service / Repository / DTO) ? »

5.2. Volet B : Infrastructure CI/CD sur la Forge GitLab de l'Université de Lille

- **Question B.1 (Capacités des Runners partagés) :**

« Quels sont les tags et les fonctionnalités activées sur les runners GitLab partagés de l'Université de Lille ? Supportent-ils le mode Docker-in-Docker (dind) pour nous permettre de construire et tester les images Docker dans le pipeline .gitlab-ci.yml, ou devons-nous exécuter les builds Maven directement dans des conteneurs d'image de base (image: maven:3.9-eclipse-temurin-21) ? »

- **Question B.2 (Limites et ressources d'exécution) :**

« Y a-t-il des quotas stricts d'espace disque, de bande passante ou de timeout d'exécution (ex. max 15 minutes) sur les pipelines d'intégration de l'université ? »

5.3. Volet C : Qualimétrie, SonarQube & Couverture de Tests

- **Question C.1 (Instance SonarQube institutionnelle) :**

« L'Université de Lille héberge-t-elle une instance centrale de SonarQube interconnectée avec la forge GitLab, avec authentification étudiante et analyse automatique des Merge Requests ? Ou devons-nous conteneuriser notre propre instance SonarQube locale sous Docker ou passer par SonarCloud ? »

- **Question C.2 (Seuil d'exigence de la Quality Gate) :**

« Dans notre Definition of Done (DoD), nous avons fixé un objectif de 80 % de couverture de code par les tests unitaires et d'intégration sur les composants métier critiques. Cette métrique correspond-elle à votre grille d'évaluation académique pour les jalons R2 et R4 ? »

5.4. Volet D : Conception et Hébergement des Simulateurs Partenaires (Mocks REST)

Le sujet exige la simulation des services partenaires (banque, horodateur municipal et transports en commun).

- **Question D.1 (Technologie préconisée pour les Mocks) :**

« Pour concevoir les simulateurs de services externes (passerelle bancaire Izli, voirie pour le stationnement et Ilévia pour les titres de bus), recommandez-vous de développer de petits microservices applicatifs dédiés (ex. sous Node.js/Express ou Spring Boot très léger), ou privilégiez-vous des outils de mock déclaratifs comme WireMock, Mockoon ou Prism (mock server OpenAPI) ? »

- **Question D.2 (Scénarios de pannes dynamiques) :**

« Attendez-vous des simulateurs qu'ils soient configurables dynamiquement pour tester la résilience et les scénarios d'anomalies (ex. simuler un refus de transaction bancaire déclenchant un rollback 2PC, ou simuler une coupure réseau de voirie) ? »

5.5. Volet E : Modalités d'Évaluation et Livrable Exact du Jalon R2 (12 Octobre 2026)

- **Question E.1 (Format officiel du rendu R2) :**

« Quelle est la forme contractuelle exacte du rendu R2 du 12 octobre : un rapport technique écrit au format PDF présentant l'analyse comparative et la justification des outils, une démonstration en direct sur machine devant vous (démonstration du `docker compose up` et du pipeline CI au vert), ou un audit asynchrone direct de notre dépôt GitLab ? »

- **Question E.2 (Attentes sur l'analyse comparative) :**

« Le dossier R2 doit-il comporter des matrices comparatives formelles d'outils (ex: Maven vs Gradle, React vs Angular, PostgreSQL vs MySQL) avec critères de notation chiffrés, ou la justification argumentée de la stack sélectionnée suffit-elle ? »

5.6. Volet F : Site de Suivi de Projet & Reporting des Temps d'Activité

Le sujet mentionne en page 6 : « Un site de suivi de projet dans lequel les enseignants pourront retrouver l'ensemble des documents [...], le reporting c-a-d les temps d'activité individuels, et la planification ».

- **Question F.1 (Emplacement et technologie du site de suivi) :**

« Quel est l'environnement d'hébergement recommandé pour le site de suivi de projet : recommandiez-vous un site statique généré via GitLab Pages (ex. avec MkDocs, Docusaurus ou VitePress) directement sur notre dépôt, l'utilisation du Wiki GitLab, ou un espace web externe dédié ? »

- **Question F.2 (Format du reporting des temps individuels) :**

« Quel formalisme attendez-vous pour le reporting des temps individuels de chaque membre : l'utilisation du système de time-tracking intégré aux issues/tickets GitLab (/spend), un tableau récapitulatif périodique consigné en annexe de chaque livrable, ou un tableur partagé ? »

6. ORGANISATION ET DÉROULEMENT DE L'ÉQUIPE EN SÉANCE

Pour refléter le niveau d'excellence et la posture d'une société d'ingénierie logicielle (Entreprise Garik), l'équipe applique une organisation stricte lors de la séance présentielle du 28 septembre.

6.1. Répartition des Prises de Parole en Séance

Membre	Pôle de Compétence	Mission & Intervention en Séance
Rayane Alli	<i>Spécialiste Outils & Lead DevOps (Scrum Master R2)</i>	Anime l'échange avec l'enseignant. Présente la démarche globale d'outillage, pose les questions du Volet B (CI/CD), Volet D (Simulateurs) et Volet E (Modalités R2).
Gautam Demeulemeester	<i>Responsable Architecture Back-Office</i>	Présente la justification technique Java 21 / Spring Boot 3.3 et pose les questions du Volet A (Validation J2E et structure Maven).
Ilyas Ait Ali	<i>Responsable Architecture Front-Office</i>	Présente la justification React 18 PWA / Tailwind CSS et la prise en compte de l'accessibilité senior (Pierre, 74 ans, RGAA AA).
Abdelkader Heddi	<i>Responsable Communication & Documentation</i>	Prend en note l'intégralité des réponses et pose les questions du Volet F (Site de suivi de projet et reporting des temps).
Khalil Bouchama	<i>Responsable Qualité (QA)</i>	Pose les questions du Volet C (SonarQube et Quality Gate de 80 %) et veille au respect des exigences DoD de la séance.

6.2. Directives de Posture Professionnelle

1. **Unité absolue devant la MOA** : Aucune hésitation ni contradiction interne entre les membres ne doit être exprimée devant l'enseignant. Les arbitrages ont été formalisés en interne dans ce guide ; l'équipe défend une vision unie et cohérente.
2. **Écoute active et reformulation** : Toute consigne ou préférence émise par l'enseignant doit être notée et reformulée en fin de séance pour confirmation.

6.3. Actions Immédiates Post-Séance

Dès la sortie de la séance du 28 septembre :

1. **Compte-rendu de séance (J+1)** : Rédaction et diffusion par Abdelkader Heddi du compte-rendu des arbitrages aux 5 membres.
2. **Enregistrement de l'ADR-021** : Formalisation de la décision d'architecture d'outillage dans `DECISIONS.md`.
3. **Mise en place sur GitLab (projet-glop-app)** :
 - Initialisation de la structure multi-dossiers (`backend/`, `frontend/`, `docker/`, `tests/`).
 - Push du premier fichier `.gitlab-ci.yml` pour valider la compilation automatique dès la semaine 1.